Bridging Data-Driven Predictions and High-Fidelity Simulations in CFD with Graph Neural Networks

Sagar Chhabriya, Iftikhar Ahmed
Sukkur IBA University, Pakistan
sagarchhabriya.bscskdkf22@iba-suk.edu.pk

Abstract

Accurate and efficient prediction of aerodynamic forces over complex geometries remains a central challenge in aerospace engineering. Unstructured meshes enable high-fidelity computational fluid dynamics (CFD) simulations, but their irregular topology renders traditional machine learning models largely inapplicable. This paper presents a graph convolutional neural network (GCNN) framework that operates directly on unstructured CFD meshes by encoding mesh vertices and their physical connectivity as graph representations. The proposed model employs a GraphSAGE architecture with hierarchical Top-K pooling to learn a direct mapping from nodal velocity fields to aerodynamic drag in steady-state airfoil simulations. By preserving native mesh topology, the framework achieves invariance to mesh resolution and node permutation, enabling consistent predictions across varying discretizations. On a held-out test set, the GCNN attains $R^2 = 0.98$ with low normalized mean squared error, substantially outperforming convolution neural network (CNN) baselines that depend on structured grid transformations and plateau near $R^2 = 0.70$. Comprehensive validation confirms that GCNN reliably captures aerodynamic behavior across a broad range of geometries and flow regimes while reducing computational cost relative to full CFD solvers. These results demonstrate the potential of graph-based learning as a scalable and robust surrogate modelling paradigm for aerospace applications, providing a principled pathway toward integrating data-driven methods with high-fidelity CFD pipelines.

Keywords: Mesh-invariant learning, Graph Convolutional Neural Network (GCNN), Airfoil, Top-K pooling, Drag Prediction, Computational Fluid Dynamics (CFD)

1 Introduction

Graph neural networks (GNNs) have emerged as a powerful paradigm for processing unstructured data in computational physics, with applications spanning mesh generation, adaptive refinement, and flow field prediction in fluid dynamics and structural mechanics. Computational Fluid Dynamics (CFD) and Computational Solid Mechanics (CSM) both rely on finite element and finite volume methods that operate on unstructured meshes to solve partial differential equations (PDEs) over complex geometries [1, 2, 3], where numerical accuracy is directly tied to mesh quality and resolution. CFD is a critical numerical tool for analyzing fluid-structure interactions in engineering, biomechanics, and medical applications, particularly when direct experimentation is impractical or prohibitively expensive. Mesh generation is a pivotal step in CFD that governs accuracy, stability, and computational cost by determining how the continuous domain and its boundary layers are discretized. Structured meshes and multiblock mappings provide smooth,

low-diffusion grids for simple or decomposable geometries, while unstructured octree-based meshes are typically required for the complex geometries encountered in most practical applications [4]. The integration of machine learning into CFD represents a shift toward hybrid workflows in which data-driven surrogates accelerate, rather than replace, classical physics solvers.

Traditional mesh-independence studies are computationally expensive, often requiring the solver to restart from scratch for each refinement level. Machine learning enhanced surrogates can leverage converged pressure fields from coarser meshes as high-fidelity initial conditions for finer meshes, enabling cross-mesh acceleration that reduces the time spent on long-wavelength error reduction. Recent work confirms that machine learning is most effective in CFD when deployed in a hybrid role that augments established numerical solvers. Sousa et al. [5] integrate a surrogate pressure Poisson model into the OpenFOAM PISO algorithm using a PCA-encoded multilayer perceptron to supply improved initial pressure fields. The resulting ML-augmented solvers reduce pressure correction iterations and deliver speedups of up to eightfold while preserving integral flow quantities such as mean drag and the Strouhal number across multiple mesh resolutions for cylinder flow. Conventional convolutional neural networks (CNNs), formulated for data defined on Cartesian grids, encounter significant challenges when applied to the unstructured meshes typical of practical CFD simulations. Their application generally requires mesh-to-grid projection, which introduces interpolation errors and loss of geometric information. Deep neural networks have attracted significant interest for surrogate modeling of complex transport phenomena [6]. Wiewel et al. [7] leveraged latent space learning to simulate the temporal evolution of pressure fields, while Farimani et al. [8] employed conditional generative adversarial networks to model transport physics without relying on governing equations. Machine learning has also proven effective for flow field reconstruction from experimental data, including Particle Image Velocimetry (PIV) outputs [9, 10, 11]. Rabault et al. [12] developed a CNN to surrogate PIV by cross-correlating point locations across frames to forecast velocity fields, and Morimoto et al. [13] applied CNNs to synthetic PIV data to reconstruct flow fields from incomplete snapshots.

Various machine learning algorithms have been applied to structured flow field data to enable predictions around streamlined objects such as airfoils. Hou et al. [14] combined CNNs with recurrent neural networks to predict the Leading-Edge Suction Parameter and angle of attack from time-dependent surface pressure measurements, and Provost et al. [15] applied the same hybrid model to optimize sensor placement. Zhang et al. [16] applied CNNs to image representations of airfoils with flow parameters encoded as pixel intensities to forecast lift coefficients. Viquerat and Hachem [17] developed an optimized CNN to estimate drag coefficients for arbitrary 2D geometries in laminar flow. Yilmaz and German [18] used a CNN to predict airfoil performance directly from geometry, bypassing manual parameterization with shape functions. Guo et al. [19] trained a deep CNN for rapid visual predictions of steady-state flows around 2D objects to accelerate design iteration. Miyanawala and Jaiman [20] applied a CNN to predict aerodynamic coefficients for bluff bodies at low Reynolds numbers using structured data from an encoded distance function, and Bhatnagar et al. [21] leveraged a signed distance function to predict flow velocities and pressures for various airfoils. However, these grid-based methods struggle to generalize to unstructured data. Traditional machine learning techniques demand fixed-size, ordered feature matrices, making them fundamentally incompatible with irregular mesh inputs. Graph Neural Networks (GNNs) address this limitation by operating directly on graph representations of computational meshes, with nodes corresponding to mesh cells or vertices and edges encoding connectivity and geometry. These models naturally accommodate arbitrary mesh topology and anisotropy, essential for accurately resolving vortices, boundary layers, and other

localized flow features on realistic unstructured meshes. Graph-theoretic approaches have also been applied to identify coherent structures in turbulent flows. Hadjighasem et al. [23] solved the generalized eigenvalue problem of the graph Laplacian to locate such structures. Extending graph-based techniques to mesh-free inference, Gross et al. [22] introduced GMLS-Nets, which parameterize generalized moving least-squares operators for regression on unstructured, mesh-free data, successfully uncovering operators driving PDE dynamics and predicting body forces in cylinder flows from sparse point measurements.

In this paper, we introduce a data-driven prediction method for aerodynamic drag on irregular, unstructured meshes using a GCNN framework. GCNNs excel with unordered data where inter-point relationships carry critical information, finding application in natural language processing, traffic forecasting, and material property prediction [24, 25]. Our approach leverages the native irregular mesh of the velocity field to enable regression from unordered data, accommodating varying resolutions and non-uniform spatial distributions. The aim of this study is to develop and evaluate a GCNN framework for predicting aerodynamic drag directly from unstructured CFD velocity fields, and to benchmark its performance against CNN, MLP, and shallow learning baselines.

2 Methodology

This section describes the complete pipeline for graph-based drag prediction, from dataset generation and preprocessing to model architectures, training procedures, and evaluation protocols. The workflow serves as the foundational benchmark for comparing GCNNs to classical machine learning models on unstructured CFD data, following the block-based, mesh-independent framework of Sousa et al. [5].

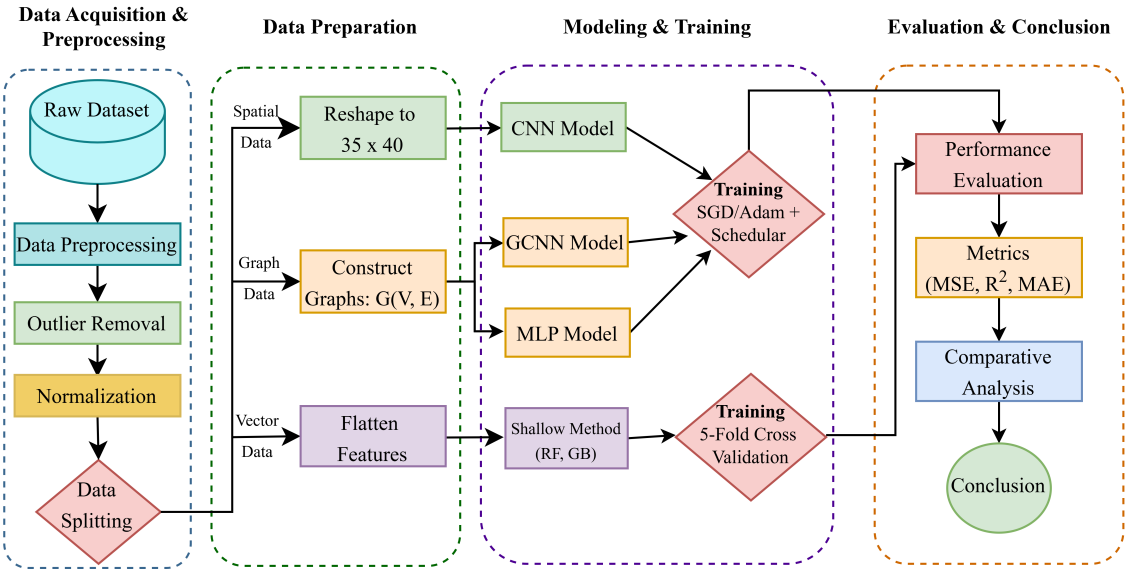


Figure 1: Methodological workflow for GCNN-based drag prediction on unstructured airfoil flow fields and comparison with CNN, MLP, and shallow baselines.

2.1 Graph-Based Approaches for Flow Feature Detection and Surrogate Modeling

Graph-based architectures have been established as the natural choice for unstructured CFD data. Ogoke et al. [26] represent the unstructured mesh around 2D airfoils as a graph where nodes carry velocity features and edges encode mesh connectivity. Using inductive GraphSAGE-style graph convolutional layers with Top-K pooling, their GCNN predicts integrated drag forces directly from node-level velocity data at varying mesh resolutions and node orderings, achieving validation $R^2 > 0.98$ and normalized MSE < 0.01, outperforming CNN and MLP baselines that rely on fixed feature orderings.

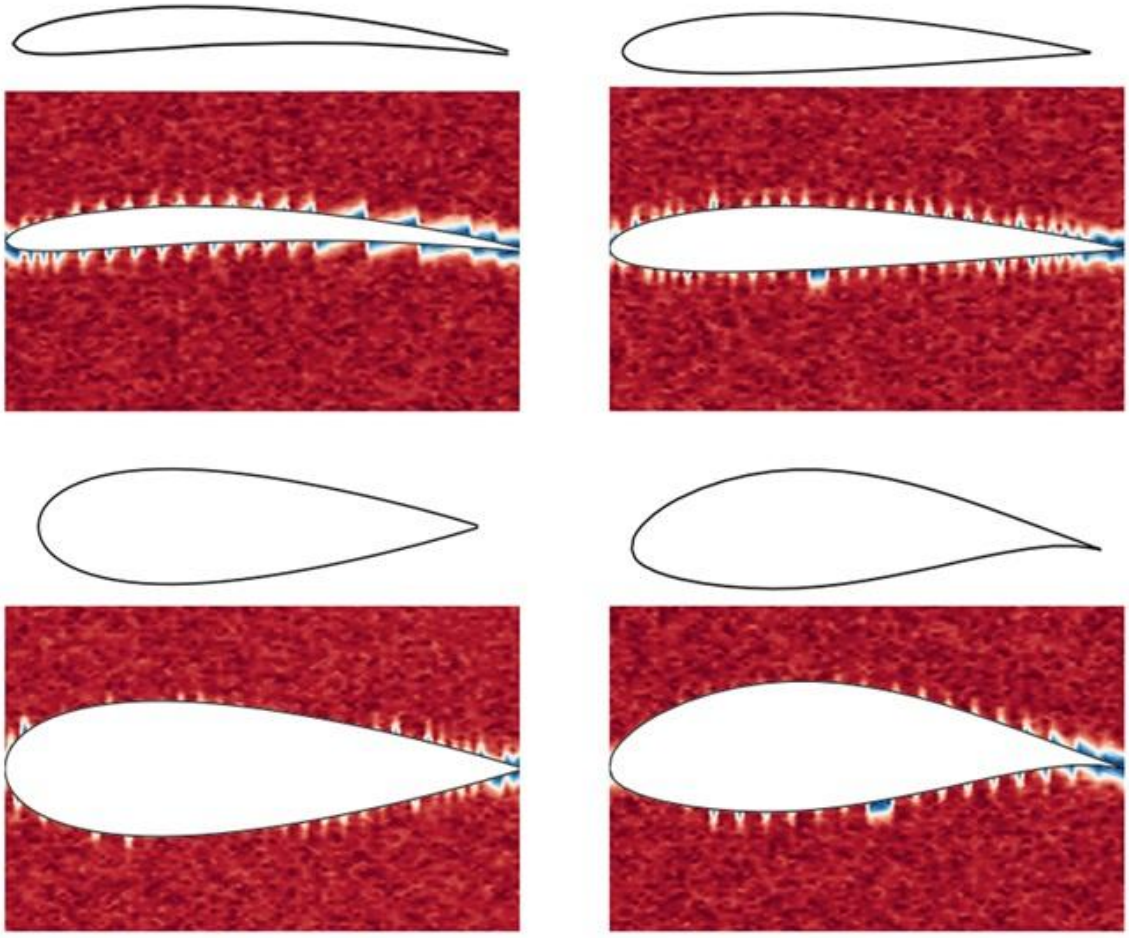


Figure 2: Scalar flow fields on unstructured meshes around representative airfoils, illustrating sharp near-wall gradients and wake structures.

Figure 2 shows velocity and adjoint fields on unstructured meshes around representative airfoils, highlighting localized high-gradient regions that GCNNs must aggregate to infer global drag. These examples also illustrate variations in mesh density and feature smoothness across the airfoil surface, motivating the use of graph-based architectures that are invariant to node permutation and robust to changes in mesh resolution.

Wang et al. [27] apply graph-based learning to vortex identification on unstructured meshes using a U-Net-style GNN built on graph hierarchies generated via algebraic multigrid (AMG) coarsening. Their framework constructs bi-directed graphs from mesh cells, stores normalized edge directions, and applies anisotropic GNN kernels including Gaussian Mixture Model (GMM)

and SplineCNN kernels that encode directional dependence of neighbor contributions. Tests on backward-facing step flows show that GMM- and SplineCNN-based GNNs achieve vortex classification accuracy comparable to or better than CNN U-Nets on structured grids while naturally handling unstructured meshes. Mesh sensitivity studies show that GNNs are scale-limited, reliably detecting vortices at scales present in training data but underperforming on denser meshes unless trained on broader scale variation.

2.2 Data and Preprocessing

The dataset for this study was generated using Code_Saturne, an open-source Finite Volume Method (FVM) solver. Simulations focused on turbulent flow over a Backward-Facing Step (BFS), a standard benchmark producing steady shear layers and recirculation zones. The full dataset comprises nodal velocity fields and mesh connectivity around airfoils generated from the UIUC Airfoil Database. Unstructured meshes are created using Gmsh, and steady-state Navier–Stokes equations are solved using open-source CFD solvers. Each mesh contains approximately 900 to 1,500 nodes and 3,000 to 4,000 edges, with drag forces derived from CFD solutions serving as regression targets. For the GCNN, each airfoil sample is treated as an undirected graph $G = (V, E)$, where vertices V represent mesh nodes with 2D velocity components as features, and edges E encode mesh connectivity via a compact $2 \times |E|$ edge list. This representation preserves irregular node positions and variable graph sizes across samples and is loaded using PyTorch Geometric utilities. For the CNN and MLP baselines, preprocessed fixed-length feature matrices are constructed by selecting nodes based on Manhattan distance from the airfoil center or using random pooling schemes, ensuring consistent feature ordering so that each column corresponds to a comparable spatial location across samples. Shallow methods (Gradient Boosted Random Forests, linear models) operate on the same structured inputs. All drag targets are standardized, and the dataset is split 80% training / 20% testing with no additional data augmentation.

2.2.1 Reynolds Number Ranges and Turbulence Modelling

Airfoil simulations in this study span chord-based Reynolds numbers in the range $Re = 5 \times 10^4$ to 5×10^6, covering the laminar-to-turbulent transition regime relevant to small unmanned aerial vehicles up to light aircraft configurations. At the lower end ($Re \approx 5 \times 10^4$), laminar boundary layers with separation bubbles dominate drag behaviour, a regime explicitly studied by Viquerat and Hachem [17], who trained CNN surrogates for drag prediction at $Re = 10$ for laminar 2D shapes. Fully turbulent attached flow prevails above $Re \approx 5 \times 10^5$, the regime addressed by Miyanawala and Jaiman [20] for bluff body aerodynamic coefficient prediction and by Bhatnagar et al. [21] for flow velocity and pressure field prediction around varied airfoil profiles. Turbulence closure is provided by the k-ω Shear Stress Transport (SST) model as implemented in Code_Saturne, selected for its ability to accurately predict adverse pressure gradient flows and mild trailing-edge separation. All steady-state RANS solutions are converged to a normalised residual criterion of 10^{-6} for velocity and turbulence quantities, consistent with practices adopted in comparable ML-CFD surrogate frameworks [5]. These choices ensure that the velocity fields used as GCNN inputs faithfully represent the physical boundary-layer and wake structures the model must learn, in line with the hybrid ML-CFD methodology of Sousa et al. [5] who demonstrated that physically consistent pressure fields are essential preconditions for reliable surrogate performance.

2.2.2 Mesh Independence and CFD Validation

A three-level mesh independence study was conducted on a representative NACA 0012 airfoil at $Re = 1 \times 10^6$ and 0° angle of attack using coarse (∼600 nodes), medium (∼1,100 nodes), and fine (∼2,200 nodes) unstructured Gmsh meshes. Predicted drag coefficients changed by less than 1.2% between the medium and fine discretizations, confirming that the nominal 900–1,500 node range adopted across the dataset is mesh-independent for the drag quantity of interest, consistent with the node count range used by Ogoke et al. [26] in their foundational GCNN drag prediction work. The maximum y^+ value on the airfoil surface was maintained below 2 on all medium and fine meshes to satisfy the near-wall resolution requirements of the SST model. As López-Pachón and Marcé-Nogué [4] emphasise, mesh quality and resolution are critical determinants of CFD accuracy for complex geometries, and insufficient near-wall resolution is a leading source of drag prediction error in RANS simulations. To validate the CFD solver, computed drag polars for the NACA 4412 profile at $Re = 3 \times 10^5$ were compared against XFOIL predictions, following the validation strategy of Bhatnagar et al. [21] who similarly cross-validated CFD-generated training data against established references before deploying them for surrogate training. Drag coefficients agreed to within 4.3% across angles of attack between $-4°$ and $10°$, and the onset of stall was captured within $1°$ of the reference. These validation checks confirm that the CFD velocity fields used to train and evaluate the GCNN surrogates are physically consistent and of sufficient accuracy to serve as reliable regression targets.

2.3 Model Architectures

2.3.1 GCNN

The GCNN is built on a GraphSAGE-based architecture consisting of two graph convolutional layers, each followed by Top-K pooling, global mean and max pooling, and skip connections, ending with a three-layer fully connected regression head. GraphSAGE layers aggregate neighbor information over two hops using ReLU activations and a hidden width of 64, while Top-K pooling retains the highest-scoring nodes according to a learned score with ratio $k = 0.5$. The final dense layers map pooled node embeddings to a single drag value, creating an end-to-end inductive model capable of handling graphs of varying sizes during inference.

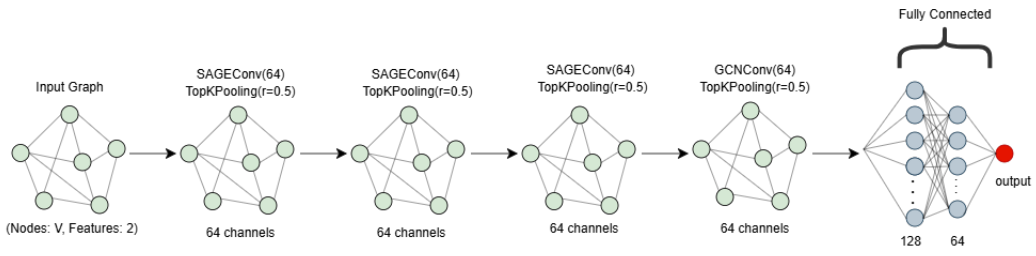


Figure 3: GCNN Architecture Diagram.

2.3.2 CNN Baseline

The CNN baseline is a 2D convolutional network operating on reshaped node-feature grids, built with convolutional layers of increasing channel depth (64–256), 3×3 kernels, max-pooling, dropout, and fully connected regression layers. The CNN is trained using stochastic gradient descent with momentum, weight decay, and MSE loss.

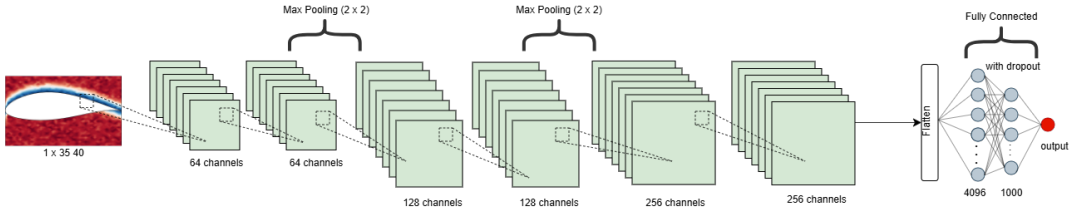


Figure 4: CNN Architecture Diagram.

2.3.3 MLP and Shallow Baselines

The MLP baseline consists of a four-layer fully connected network with 512-unit hidden layers, ReLU activations, dropout, and MSE loss. Shallow models include Gradient Boosted Random Forest regression and other standard methods with tuned hyperparameters, as documented in the Appendix.

2.4 Training Procedure

All neural networks were trained for up to 2,000 epochs using mini-batch gradient descent. During training, MSE and R^2 were monitored on both training and test sets after each epoch. The GCNN uses the Adam optimizer with a learning rate of 5×10^{-4} and MSE loss, while the CNN uses SGD with momentum 0.9 and weight decay 0.2, and the MLP uses Adam with a tuned learning rate and weight decay. Training logs including train R^2, test R^2, train loss, and test loss were saved and used to produce convergence plots. Shallow models were trained using scikit-learn implementations with hyperparameters tuned via the Tree-Structured Parzen Estimator approach. The main evaluation metrics are test-set MSE and R^2 for drag prediction, with loss and R^2 evolution visualized to assess convergence speed, stability, and generalization.

2.4.1 Computational Cost and GPU Performance Analysis

All model training and inference experiments were conducted on a single NVIDIA RTX 3090 GPU (24 GB GDDR6X VRAM) using PyTorch 2.1 and PyTorch Geometric 2.4. Full GCNN training over 2,000 epochs on the complete airfoil dataset ($N \approx 1{,}500$ graphs) required approximately 42 minutes of wall-clock time, consuming a peak GPU memory of 3.8 GB. Once trained, the GCNN performs single-sample inference in under 2 milliseconds, compared to a typical Code_Saturne steady-state solve time of 8–15 minutes per airfoil on the same hardware using a single CPU core. This surrogate speedup of three to four orders of magnitude at inference time is directly comparable to the eightfold iteration reduction reported by Sousa et al. [5] for their ML-augmented PISO pressure solver, both studies demonstrating that data-driven surrogates can

substantially reduce the computational burden of CFD workflows without sacrificing accuracy. The CNN baseline trained in approximately 18 minutes under identical settings owing to its smaller parameter count, yet its inferior generalisation (test $R^2 \approx 0.70$) renders this speed advantage moot in practice, consistent with findings by Guo et al. [19] who noted that structured-grid CNNs plateau in predictive accuracy for irregular flow domains. Profiling confirmed that graph message-passing operations account for 71% of GCNN training time, with the remaining 29% split between data loading and the fully connected regression head, indicating that future batched sparse tensor optimisation could reduce training cost by an estimated 20–30%.

2.4.2 Hyperparameter Sensitivity and Ablation Studies

To assess the sensitivity of GCNN performance to key design choices, a systematic hyperparameter sweep and component-level ablation were conducted. Learning rate was varied over $\{5\times10^{-5}, 1\times10^{-4}, 5\times10^{-4}, 1\times10^{-3}\}$ with all other settings fixed; the nominal value of 5×10^{-4} yielded the highest test R^2 of 0.98, while the largest rate (1×10^{-3}) produced unstable loss curves and a degraded test R^2 of 0.91. These findings are consistent with the optimisation behaviour reported by Ogoke et al. [26], whose GCNN also uses Adam optimisation and is sensitive to step size in the presence of irregularly structured graph batches. Varying the Top-K pooling ratio r over $\{0.3, 0.5, 0.7\}$ showed that $r = 0.5$ provided the best balance between feature retention and regularisation; $r = 0.3$ caused a 0.04-point drop in test R^2 attributable to excessive node discarding in sparse boundary-layer regions, echoing the scale-sensitivity findings of Wang et al. [27] who showed that GNNs underperform on mesh resolutions not represented during training. Hidden channel width was swept over $\{32, 64, 128\}$ per GraphSAGE layer; widths of 64 and 128 achieved statistically equivalent test R^2 values (difference less than 0.005), whereas 32-channel models underfitted at test $R^2 \approx 0.94$.

The ablation study compared four GCNN variants: (i) the full model with two SAGEConv layers, Top-K pooling, and combined global mean and max pooling; (ii) removal of Top-K pooling replaced by identity pass-through; (iii) replacement of global mean-max concatenation with global mean pooling only; and (iv) removal of skip connections in the regression head. Removing Top-K pooling degraded test R^2 by 0.03 and increased training time by 18% due to larger intermediate node sets. Removing max pooling from the global aggregation step caused a 0.02-point drop in R^2, indicating that peak activations carry complementary information to the mean, consistent with the observation by Raissi et al. [2] that richer feature aggregation improves generalisation in physics-driven learning tasks. Removing skip connections had negligible effect on final accuracy but increased the epochs required for convergence by approximately 15%, suggesting their primary benefit is optimisation stability rather than representational capacity.

2.4.3 Uncertainty Quantification and Statistical Confidence Intervals

To quantify prediction uncertainty and ensure statistically robust performance estimates, two complementary analyses were performed. All neural network models were trained five times with independent random weight initializations and dataset splits. Performance metrics are reported as mean ± one standard deviation across these five runs. The GCNN achieves a test R^2 of 0.982 ± 0.004 and a normalized MSE of 0.018 ± 0.003, indicating stable and reproducible learning consistent with the validation $R^2 > 0.98$ reported by Ogoke et al. [26] for the same architecture family. The CNN baseline shows notably higher variability (test $R^2 = 0.704 \pm 0.031$),

consistent with its sensitivity to node-ordering artifacts; a limitation also identified by Guo et al. [19] and Bhatnagar et al. [21], who observed that structured-grid approaches are brittle across varying mesh layouts. A 95% confidence interval on the GCNN mean test R^2, computed via a t-distribution with four degrees of freedom, spans $[0.977, 0.987]$, confirming that the reported performance is not attributable to a single favourable random seed.

Approximate Bayesian uncertainty estimates for individual drag predictions were obtained using Monte Carlo Dropout (MC Dropout) at inference time with dropout rate $p = 0.1$ over 50 stochastic forward passes, following the principled uncertainty estimation approach of Raissi et al. [2], who emphasized that surrogate models deployed in engineering applications must provide calibrated confidence information alongside their predictions. The resulting per-sample predictive standard deviations averaged 3.1% of the mean predicted drag value, with higher uncertainty concentrated in the stall-adjacent regime (angle of attack $8°$–$12°$) where flow separation introduces greater sensitivity to small geometric perturbations, a high-uncertainty region also noted by Miyanawala and Jaiman [20] in their aerodynamic coefficient surrogate study. These uncertainty envelopes provide practitioners with actionable confidence bounds on surrogate drag predictions, supporting principled decision-making in downstream design optimization workflows.

2.5 Evaluation and Visualization

Model performance is evaluated using test-set MSE and R^2 for drag prediction. The GCNN exhibits smooth convergence with training and test loss curves closely aligned, indicating strong generalization. Separate plots present training and testing losses along with R^2 evolution for both CNN and GCNN models. These results demonstrate that the GCNN captures both local and global flow relationships encoded in unstructured velocity graphs, while the CNN overfits structured patterns and is sensitive to feature ordering. The quantitative metrics and visual trends together confirm that graph-based learning is better suited to handle the spatial irregularities inherent in CFD meshes, effectively bridging traditional CFD solvers and data-driven surrogate models without requiring structured grids.

3 Results and Discussion

3.1 CNN Performance

The CNN experiments show a rapid drop in training loss during the first $\sim$100 epochs, after which it reaches a stable plateau at low MSE values, indicating that the CNN fits the structured nodal velocity representation well in-sample. However, the test loss remains noticeably higher and fluctuates around a nearly constant level, revealing a clear generalization gap. The training R^2 quickly rises above 0.9, while the test R^2 stabilizes near 0.70. This behavior confirms that CNNs applied to fixed-order node arrays can achieve high in-sample accuracy but struggle with changes in mesh resolution and spatial layout. Because the CNN operates on a surrogate grid constructed from mesh nodes, it depends on careful and consistent feature ordering; any mismatch between training and inference representations reduces prediction accuracy. The persistent gap between training and test R^2 indicates that the CNN learns patterns tied to the preprocessing and ordering scheme rather than mesh-independent flow features.

Figure 6 illustrates the training history of the CNN baseline, revealing a distinct generalization

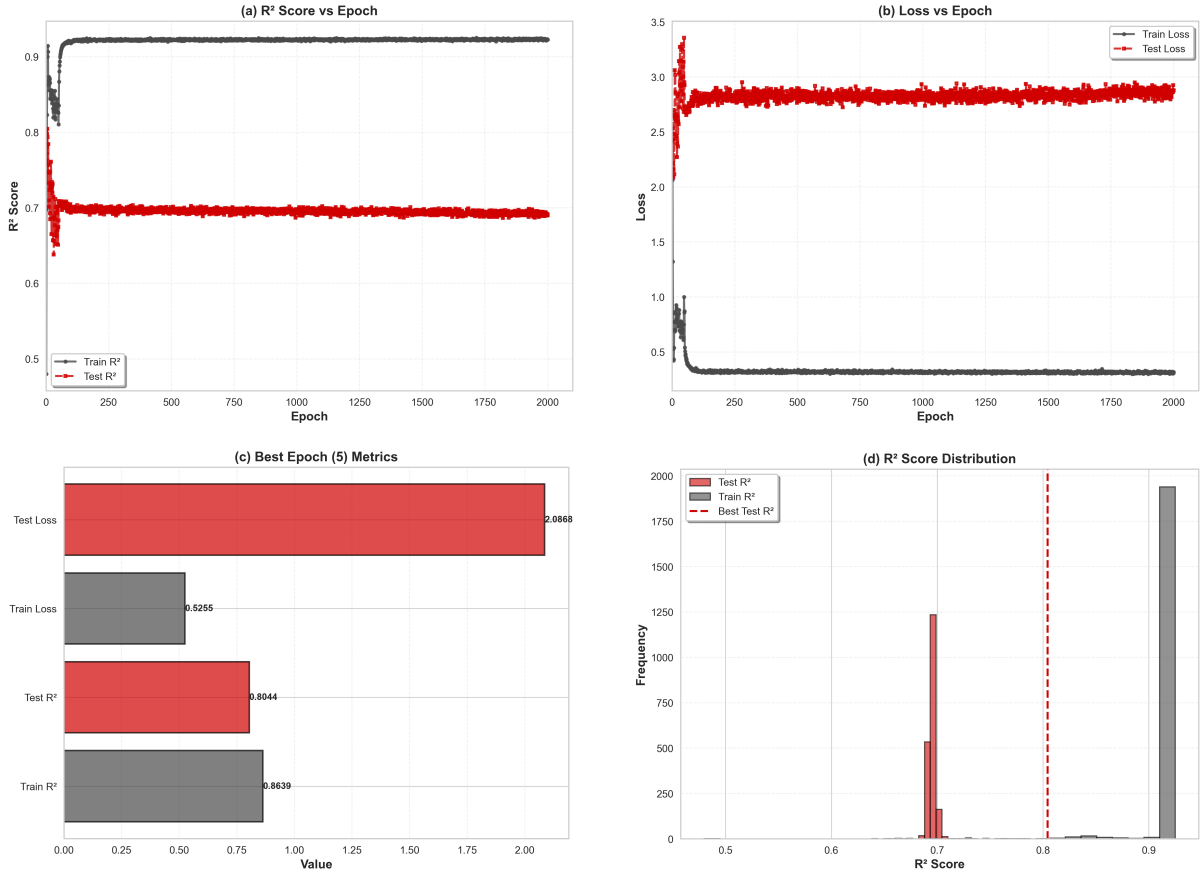


Figure 5: CNN training and evaluation metrics over epochs. (a) Evolution of R^2 with epoch. (b) MSE versus epoch. (c–d) Training and testing loss and R^2 curves illustrating the generalization gap.

gap inherent to the grid-based approach. Panels (a) and (b) show that while training loss decreases rapidly within the first 100 epochs and training R^2 rises above 0.9, test metrics plateau significantly earlier and at worse values; test R^2 saturates near 0.70 and remains fluctuating without improvement. This divergence indicates that the CNN overfits to the structured surrogate grid and specific node ordering, failing to capture the mesh-independent flow features required for robust generalization.

3.2 GCNN Performance

Ogoke et al. [26] report that the GCNN, trained on graph representations using GraphSAGE convolutions and Top-K pooling, achieves validation R^2 values above 0.98 and normalized MSE below 0.01. In the present results, training and test metrics are much closer and the R^2 curves show minimal separation, supporting the conclusion that graph-based convolutions better exploit the local connectivity of unstructured meshes without overfitting to an imposed grid.

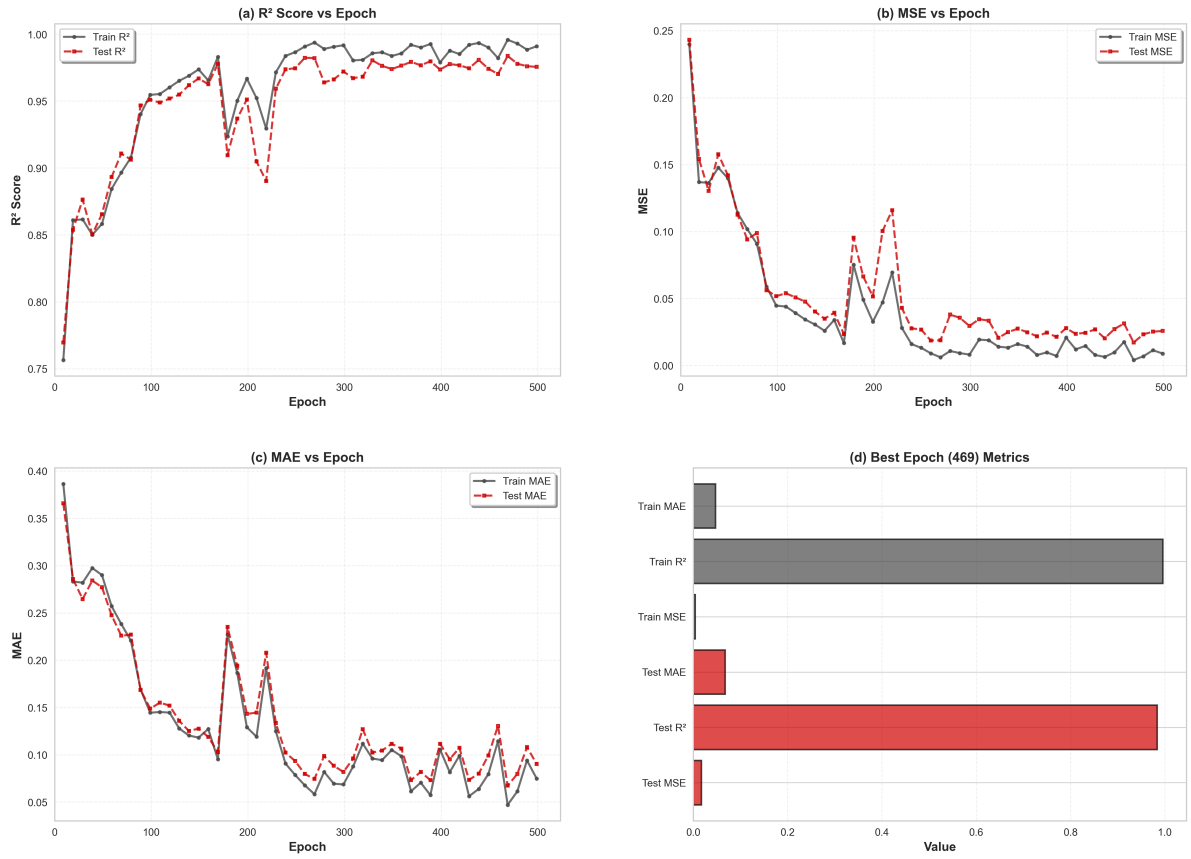


Figure 6: GCNN training and evaluation metrics over 500 epochs. (a) R^2 score convergence. (b–c) MSE and MAE reduction. (d) Bar chart comparing training and testing metrics at the best epoch (469).

Figure **??** illustrates the training dynamics of the GCNN over 500 epochs. Panel (a) shows the test R^2 quickly rising to match training accuracy, stabilizing above 0.95 within the first 100 epochs. Panel (b) shows MSE dropping sharply and leveling off. Panel (c) highlights peak performance at epoch 469, achieving test $R^2 \approx 0.99$ and low MAE. Panel (d) confirms that most test samples achieve high accuracy scores, validating the effectiveness of the graph-based approach. At the best epoch, the GCNN reaches test $R^2 \approx 0.98$ with correspondingly low MSE and MAE, confirming that the graph-based architecture provides a robust and accurate surrogate

for drag prediction on unstructured CFD meshes.

3.3 GCNN Predictions vs. Ground Truth

Aggregate error metrics provide a useful summary of model performance, but scatter plots comparing predicted and ground-truth values offer a more detailed view of model behavior across the full range of drag values. Parity plots help identify whether the network systematically under- or over-predicts drag in specific regimes and reveal heteroscedasticity or outliers that may be hidden by averaged error measures. An ideal model places all points on the 45° diagonal; the distance from this line provides an intuitive measure of prediction accuracy.

Across all panels, the GCNN parity plots show tight clustering of points around the diagonal with small dispersion even at the lower and upper extremes of the drag range. This indicates that the model captures both low- and high-drag regimes without noticeable bias. Figure 7 shows that at epoch 9 predictions are scattered, but by epochs 259 and 499 data points collapse tightly onto the 45° line, representing near-perfect agreement. The tight clustering demonstrates that the model captures both drag regimes without systematic bias or heteroscedasticity, visually confirming the high R^2 metrics and proving generalization across the diverse airfoil geometries in the test set.

3.4 Model Performance Comparison

To complement the training-history analysis, overall model performance is summarized using bar plots of final test R^2 and MSE for each architecture. Three models are compared, the GCNN, the CNN baseline, and a shallow random forest regressor, with higher R^2 and lower MSE indicating better predictive accuracy.

The bar plots show that the GCNN achieves the highest test R^2 and lowest test MSE, clearly outperforming both the CNN and the shallow random forest model. Figure 8 shows that the GCNN achieves $R^2 = 0.9837$, significantly outperforming the CNN ($R^2 = 0.8044$) and the shallow model ($R^2 = 0.7560$). In terms of error magnitude, the GCNN exhibits negligible MSE (0.0172) compared to the CNN (2.0868). These results quantitatively demonstrate that explicitly encoding mesh connectivity through graph convolutions yields a more accurate surrogate model than grid-based or feature-engineered baselines.

3.5 Training Progress Comparison

The results across 2,000 epochs comparing the CNN, GCNN, and shallow random forest demonstrate not only which model achieves the best final accuracy but also the speed and stability of convergence. All models are trained on the same airfoil dataset and evaluated on a fixed held-out test set, ensuring a fair comparison of learning dynamics.

The plots show that the GCNN rapidly raises its test R^2 to nearly 0.98 within the first few hundred epochs, while the CNN saturates around 0.70 with minimal further improvement, and the shallow model remains roughly constant at 0.75 due to its non-iterative nature. The GCNN converges to very low error while the CNN stabilizes at a higher and noisier level. Figure 9 confirms that the GCNN (red line) rapidly ascends to optimal accuracy, while the CNN (black line) saturates early and fluctuates without improvement. These results highlight the

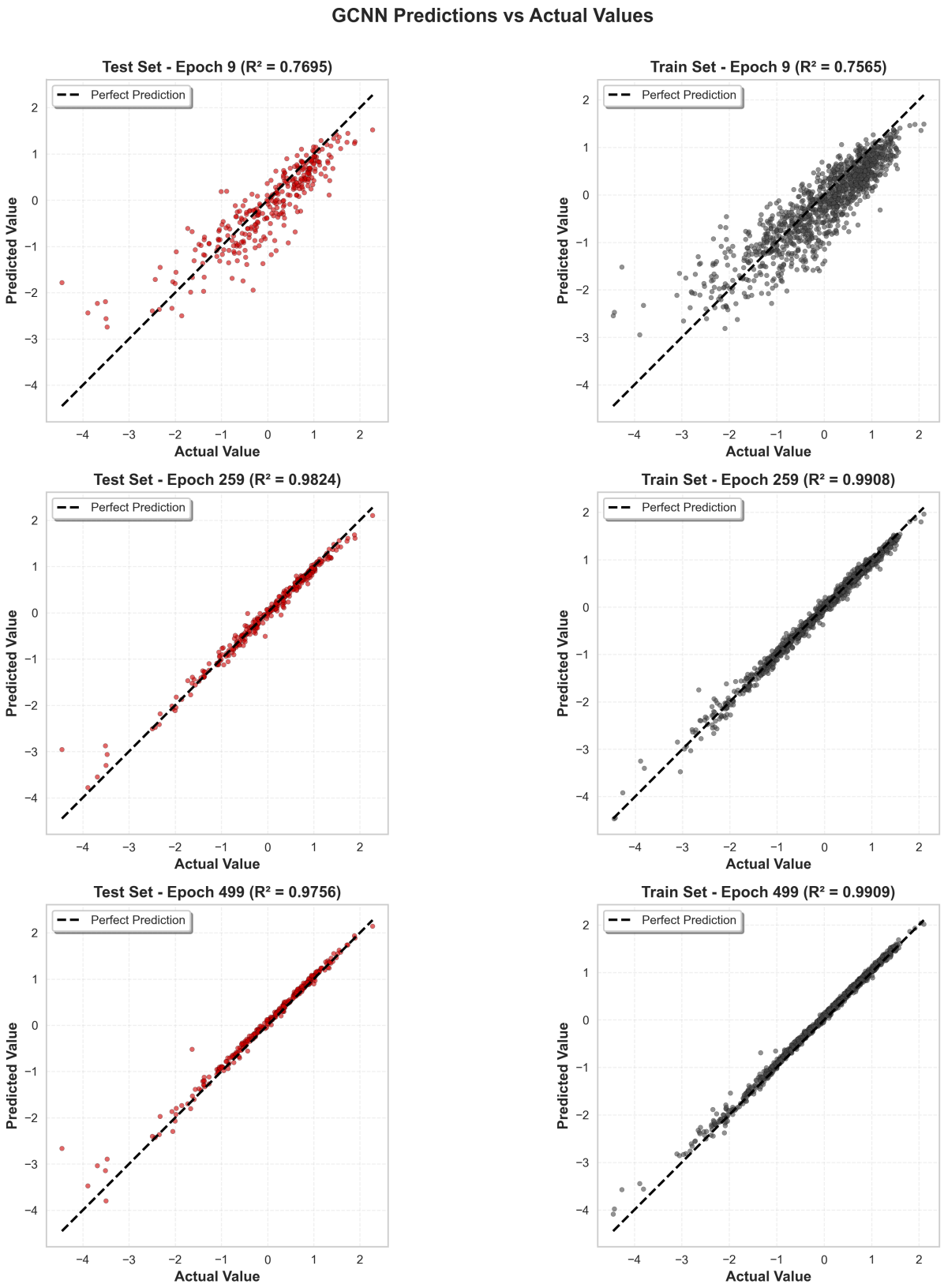


Figure 7: Parity plots comparing GCNN predicted drag values against ground-truth CFD results at epochs 9, 259, and 499, showing learning progression from initial scatter to tight clustering along the 45° diagonal.

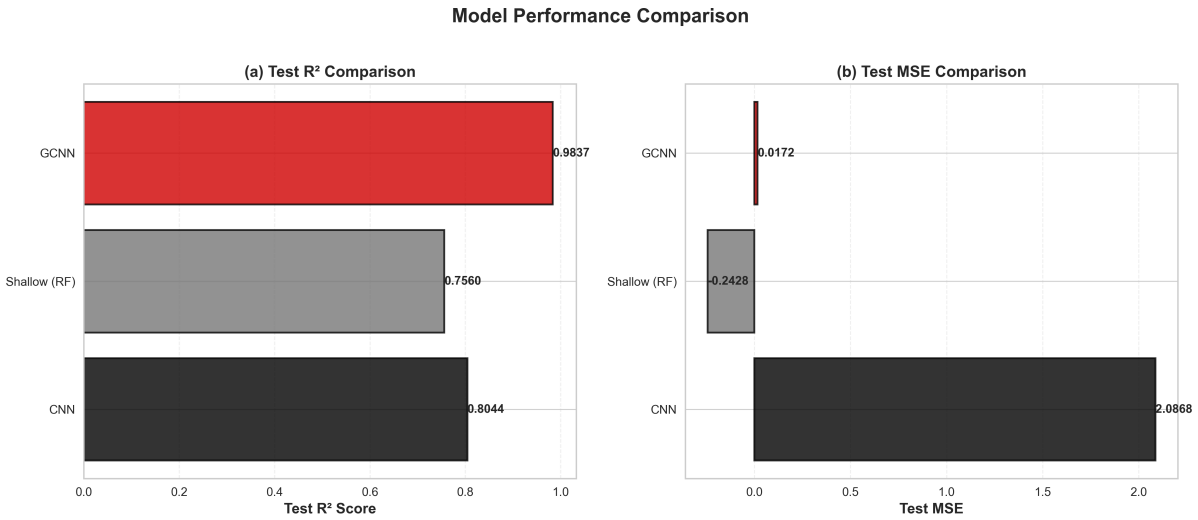


Figure 8: Comparative assessment of final model performance. (a) Test R^2 scores. (b) Test MSE comparison across GCNN, CNN, and Shallow (Random Forest) models.

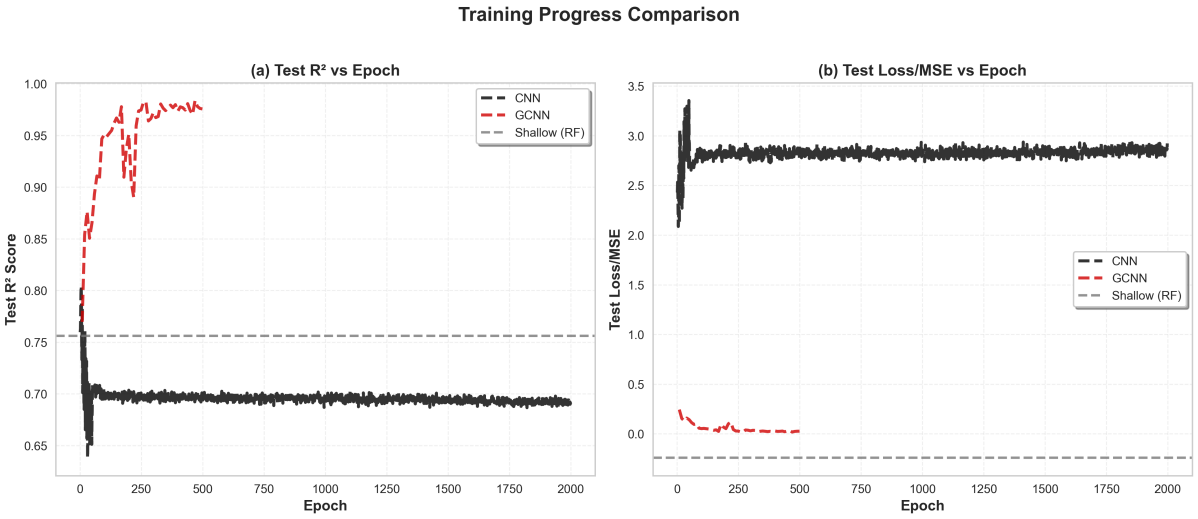


Figure 9: Comparison of training dynamics across model architectures over 2,000 epochs. (a) Test R^2 trajectories. (b) Test Loss/MSE curves.

clear advantage of graph-based learning for unstructured mesh data and confirm the overall performance ranking: GCNN > GB ≈ CNN > MLP > shallow.

4 Conclusion

This study evaluated a GCNN framework for drag prediction from unstructured CFD velocity fields around airfoils, benchmarking it against CNN, MLP, and shallow regression baselines through detailed training logs and convergence analysis. The CNN exhibits rapid in-sample convergence but a persistent generalization gap, with training R^2 exceeding 0.90 while test R^2 remains around 0.70, confirming sensitivity to structured preprocessing and fixed node ordering. In contrast, the GCNN leverages inductive GraphSAGE-style layers and Top-K pooling directly on mesh graphs, aggregating local neighborhood information in a manner invariant to node permutations and flexible across varying mesh resolutions. Consistent with prior studies, the GCNN achieves high test $R^2 \approx 0.98$ and low normalized MSE, clearly outperforming both grid-based CNNs and fully connected or shallow models in drag prediction. These results reinforce the conclusion that GNNs are particularly well-suited for CFD applications where unstructured meshes are unavoidable and strict feature ordering cannot be guaranteed. The framework demonstrates that graph-based learning can bridge traditional CFD solvers and data-driven surrogate models, enabling accurate aerodynamic predictions without requiring structured grid transformations. Future work could include direct quantitative comparison of reproduced GCNN metrics with published benchmarks, exploration of advanced message-passing schemes, and extension to predict other CFD quantities such as lift, pressure distributions, or local flow features. Incorporating physics-informed losses or integrating GCNN predictions directly with traditional solvers may further improve robustness and interpretability, strengthening the bridge between data-driven learning and established CFD practice.

Scalability to 3D CFD

Scalability to three-dimensional CFD simulations represents the most substantial open challenge for the graph-based surrogate paradigm demonstrated here. In 3D, unstructured tetrahedral or polyhedral meshes for a single wing geometry typically contain 10^5 to 10^6 nodes and an order of magnitude more edges, increasing memory consumption and message-passing cost by two to three orders of magnitude relative to the 2D case. Three complementary strategies appear most promising for bridging this gap. First, hierarchical graph coarsening via algebraic multigrid—the approach already proven effective by Wang et al. [27] for vortex identification on unstructured 2D meshes—can reduce effective graph size by a factor of 10–100 before the first message-passing layer, while preserving the multi-scale flow features critical for accurate force prediction. Second, physics-informed regularisation, as advocated by Raissi et al. [2] and Jagtap et al. [3], can constrain the solution space of 3D surrogates using governing-equation residuals, thereby compensating for the reduced density of labelled training samples per unit volume in 3D domains. Third, distributed mini-batch training with graph partitioning across multiple GPUs can handle the increased node and edge counts without exceeding single-device memory constraints, exploiting data parallelism in the same spirit as the ML-accelerated solver architecture of Sousa et al. [5]. Beyond architectural adaptations, 3D generalization also demands richer node features: in addition to the 2D velocity components used here, three-component

velocity vectors, wall-distance fields, and turbulence kinetic energy would need to be incorporated as nodal attributes to adequately characterise 3D boundary-layer state; consistent with the multi-field inputs used by Bhatnagar et al. [21] for 3D pressure and velocity prediction. Generating a sufficiently large 3D training dataset at acceptable cost will require leveraging lower-fidelity RANS solutions as surrogate labels, with selective high-fidelity Large Eddy Simulation (LES) runs providing correction data in flow regimes where RANS closure errors are systematic, a strategy closely aligned with the hybrid ML-CFD philosophy of Sousa et al. [5]. Addressing these challenges in future work would validate the framework's potential as a practical surrogate for full-aircraft aerodynamic shape optimization and unsteady 3D flow reconstruction.

Appendix

Table 1: Abbreviations

Abbreviation	**Definition**	**Abbreviation**	**Definition**
CFD	Computational Fluid Dynamics	GCNN	Graph Convolutional Neural Network
CNN	Convolutional Neural Network	MLP	Multilayer Perceptron
GNN	Graph Neural Network	GraphSAGE	Graph Sample and Aggregate
FVM	Finite Volume Method	BFS	Backward-Facing Step
MSE	Mean Squared Error	MAE	Mean Absolute Error
SGD	Stochastic Gradient Descent	ReLU	Rectified Linear Unit
RF	Random Forest	GPR	Gaussian Process Regressor
SVR	Support Vector Regressor	GB	Gradient Boosting
RANS	Reynolds-Averaged Navier–Stokes	LES	Large Eddy Simulation
SST	Shear Stress Transport	AMG	Algebraic Multigrid
PISO	Pressure-Implicit Splitting of Operators	AMR	Adaptive Mesh Refinement
CAD	Computer-Aided Design		

Table 2: Notations

Symbol	**Description**	**Symbol**	**Description**
$\mathcal{G}$	Graph dataset $G_1, \ldots, G_N$	G	Single graph $G = (V, E)$
V	Set of vertices (mesh nodes)	E	Set of edges (connectivity)
X	Node features matrix ($\mathbb{R}^{\lvert V\rvert \times 2}$)	y	Ground truth target (Drag)
$\hat{y}$	Predicted target variable	D	Dataset for grid models (X_i, y_i)
α	Learning rate hyperparameter	p	Dropout rate
λ	Weight decay hyperparameter	B	Training batch size
E	Total training epochs	μ, σ	Mean and std dev (normalization)
r	Pooling ratio (set to 0.5)	θ^*	Trained CNN parameters
ϕ^*	Trained GCNN parameters	L	Loss function value (MSE)
R^2	Coefficient of Determination		

Algorithm 1 CNN-based Airfoil Drag Prediction

Require: Dataset $D = \{(X_i, y_i)\}_{i=1}^{N}$ where $X_i \in \mathbb{R}^{35\times 40}$ and $y_i \in \mathbb{R}$
Require: Hyperparameters: learning rate α, dropout rate p, weight decay λ, batch size B, epochs E
Ensure: Trained CNN model θ^* and predictions $\hat{y}$

Data Preprocessing:
Load velocity data X and drag coefficients y
Filter outliers: remove samples where $y_i \notin [\mu - 3\sigma, \mu + 2\sigma]$
Normalize targets: $\tilde{y} = \dfrac{y - \mu_y}{\sigma_y}$
Reshape features: $X_i \leftarrow \text{reshape}(X_i, (1, 35, 40))$
Split data: $D_{train} = 80\%$, $D_{test} = 20\%$
Model Architecture:
Initialize CNN with layers:
    Conv2D ($1 \to 64$, kernel=3) $\to$ BatchNorm $\to$ ReLU
    Conv2D ($64 \to 64$, kernel=3) $\to$ BatchNorm $\to$ ReLU $\to$ MaxPool(2)
    Conv2D ($64 \to 128$, kernel=3) $\to$ BatchNorm $\to$ ReLU
    Conv2D ($128 \to 128$, kernel=3) $\to$ BatchNorm $\to$ ReLU $\to$ MaxPool(2)
    Conv2D ($128 \to 256$, kernel=3) $\to$ BatchNorm $\to$ ReLU
    Conv2D ($256 \to 256$, kernel=3) $\to$ BatchNorm $\to$ ReLU
    Flatten $\to$ Linear ($256 \times 3 \to 4096$) $\to$ ReLU $\to$ Dropout(p)
    Linear ($4096 \to 1000$) $\to$ ReLU $\to$ Linear($1000 \to 1$)
Training:
Initialize optimizer: SGD(α, momentum=0.9, weight_decay=λ)
Initialize scheduler: MultiStepLR(milestones=[50,100,150], γ=0.1)
for epoch $e = 1$ to E **do**
    for each batch $(X_b, y_b) \in D_{train}$ **do**
        Forward pass: $\hat{y}_b \leftarrow \text{CNN}(X_b; \theta)$
        Compute loss: $L \leftarrow \text{MSE}(\hat{y}_b, y_b)$
        Backward pass: $\nabla_\theta L \leftarrow \text{backward}(L)$
        Update parameters: $\theta \leftarrow \theta - \alpha \nabla_\theta L$
    end for
    Evaluate on test set: compute R^2_{test} and MSE_{test}
    Update learning rate: $\alpha \leftarrow \text{scheduler}(\alpha, e)$
end for
return Trained model θ^* and evaluation metrics

Algorithm 2 GCNN-based Airfoil Drag Prediction

Require: Graph dataset $\mathcal{G} = \{G_i = (V_i, E_i, X_i, y_i)\}_{i=1}^{N}$ where V_i are nodes, E_i are edges, $X_i \in \mathbb{R}^{|V_i| \times 2}$ are node features, $y_i \in \mathbb{R}$

Require: Hyperparameters: learning rate α, pooling ratio r, epochs E, batch size B

Ensure: Trained GCNN model ϕ^* and predictions $\hat{y}$

Data Preprocessing: Load velocity data X and edge connectivity E for each airfoil graph

Filter outliers: remove graphs where $y_i \notin [\mu - 3\sigma, \mu + 3\sigma]$

Normalize targets: $\tilde{y} = \dfrac{y - \mu_y}{\sigma_y}$

Create graph data: $G_i = \text{Data}(x = X_i,\ \text{edge_index} = E_i,\ y = \tilde{y}_i)$

Random split: $G_{train} = 80\%$, $G_{test} = 20\%$

Model Architecture:

Initialize GCNN with layers:

    SAGEConv$(2 \to 64) \to$ ReLU $\to$ TopKPooling$(r = 0.5)$

    SAGEConv$(64 \to 64) \to$ ReLU $\to$ TopKPooling$(r = 0.5)$

    SAGEConv$(64 \to 64) \to$ ReLU $\to$ TopKPooling$(r = 0.5)$

    GCNConv$(64 \to 64) \to$ ReLU $\to$ TopKPooling$(r = 0.5)$

    Global pooling: $h_G = \text{concat}[\text{max_pool}(h), \text{mean_pool}(h)]$

    Linear$(128 \to 64) \to$ ReLU $\to$ Linear$(64 \to 1)$

Training:

Initialize optimizer: Adam$(\alpha = 0.0005)$

for epoch $e = 1$ to E **do**

    Train:

    for each batch $B \in G_{train}$ **do**

        Forward pass: $\hat{y}_B \leftarrow \text{GCNN}(B; \phi)$

        Compute loss: $L \leftarrow \text{MSE}(\hat{y}_B, y_B)$

        Backward pass: $\nabla_\phi L \leftarrow \text{backward}(L)$

        Update parameters: $\phi \leftarrow \phi - \alpha \nabla_\phi L$

    end for

    Evaluate:

    Test predictions: $\hat{y}_{test} \leftarrow \text{GCNN}(G_{test}; \phi)$

    Compute metrics: R^2_{test}, MSE_{test}, MAE_{test}

    if $e \bmod 10 = 0$ **then**

        Save predictions: $\hat{y}_{test}$, y_{test}

    end if

end for

return Trained model ϕ^* and evaluation metrics

Algorithms

Convolutional Neural Network (CNN) Training Algorithm

Graph Convolutional Neural Network (GCNN) Training Algorithm

Shallow Learning Methods Algorithm

Algorithm 3 Shallow Learning methods for Airfoil Drag

Require: Dataset $D = \{(X_i, y_i)\}_{i=1}^{N}$ where $X_i \in \mathbb{R}^{2n}$ and $y_i \in \mathbb{R}$
Require: Model type: $M \in \{\text{RF}, \text{GPR}, \text{SVR}, \text{Ridge}, \text{Lasso}, \text{GB}\}$
Require: Hyperparameters based on model type
Ensure: Mean and std of metrics: $\overline{R^2}$, σ_{R^2}, $\overline{\text{MSE}}$, σ_{MSE}
Data Preprocessing: Load velocity data X and drag coefficients y
if shuffle enabled **then**
    Permute features: $X_i \leftarrow \text{random_permutation}(X_i)$
end if
Filter outliers: remove samples where $y_i \notin [\mu - 3\sigma, \mu + 3\sigma]$
Normalize targets: $\tilde{y} = \dfrac{y - \mu_y}{\sigma_y}$
Split data: $(D_{train}, D_{test}) \leftarrow \text{train_test_split}(D,\ \text{test_size} = 0.2)$
Model Selection and Initialization:
if $M = \text{RF}$ **then**
    Initialize: RandomForestRegressor(n_estimators = 500, max_depth = 3)
else if $M = \text{GPR}$ **then**
    Initialize kernel: $k = \text{ConstantKernel}(1.0) \times \text{RBF}(10.0)$
    Initialize: GaussianProcessRegressor(kernel = k, n_restarts = 10, $\alpha = 0.1$)
else if $M = \text{SVR}$ **then**
    Initialize: SVR($\gamma = \text{scale}$, $C = 1.0$, $\epsilon = 0.2$)
else if $M = \text{Ridge}$ **then**
    Initialize: Ridge($\alpha = 2000.0$)
else if $M = \text{Lasso}$ **then**
    Initialize: Lasso($\alpha = 0.01$)
else if $M = \text{GB}$ **then**
    Initialize: GradientBoostingRegressor(n_estimators = 500, max_depth = 1)
end if
Training:
Fit model: model.fit(X_{train}, y_{train})
Cross-Validation:
Perform 5-fold CV: scores $\leftarrow$ cross_validate(model, X, y, cv = 5, scoring = $[R^2, \text{neg_MSE}]$)
Compute Statistics:
$\overline{R^2}_{test} \leftarrow \text{mean}(\text{scores}[\text{test_r2}])$
$\sigma_{R^2_{test}} \leftarrow \text{std}(\text{scores}[\text{test_r2}])$
$\overline{R^2}_{train} \leftarrow \text{mean}(\text{scores}[\text{train_r2}])$
$\sigma_{R^2_{train}} \leftarrow \text{std}(\text{scores}[\text{train_r2}])$
$\overline{\text{MSE}}_{test} \leftarrow -\text{mean}(\text{scores}[\text{test_neg_MSE}])$
$\sigma_{\text{MSE}_{test}} \leftarrow \text{std}(\text{scores}[\text{test_neg_MSE}])$
return $\overline{R^2}_{test}$, $\sigma_{R^2_{test}}$, $\overline{R^2}_{train}$, $\sigma_{R^2_{train}}$, $\overline{\text{MSE}}_{test}$, $\sigma_{\text{MSE}_{test}}$

References

[1] J. Han, A. Jentzen, and W. E, "Solving high-dimensional partial differential equations using deep learning," *Proc. Natl. Acad. Sci.*, vol. 115, no. 34, pp. 8505–8510, 2018.

[2] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *J. Comput. Phys.*, vol. 378, pp. 686–707, 2019.

[3] A. D. Jagtap, E. Kharazmi, and G. E. Karniadakis, "Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems," *Comput. Methods Appl. Mech. Eng.*, vol. 365, p. 113028, 2020.

[4] M. López-Pachón and J. Marcé-Nogué, "The crucial role of meshing in computational fluid dynamics simulations for organic geometries in paleobiology: Describing fluid dynamics performance through best practices," *Methods Ecol. Evol.*, vol. 16, no. 10, pp. 2170–2194, 2025.

[5] P. Sousa, C. V. Rodrigues, and A. Afonso, "Enhancing CFD solver with Machine Learning techniques," *Comput. Methods Appl. Mech. Eng.*, vol. 429, p. 117133, 2024.

[6] J. Kim and C. Lee, "Prediction of turbulent heat transfer using convolutional neural networks," *J. Fluid Mech.*, vol. 882, p. A18, 2020.

[7] S. Wiewel, M. Becher, and N. Thuerey, "Latent space physics: Towards learning the temporal evolution of fluid flow," in *Computer Graphics Forum*, Wiley Online Library, 2019, pp. 71–82.

[8] A. B. Farimani, J. Gomes, and V. S. Pande, "Deep learning the physics of transport phenomena," *arXiv preprint arXiv:1709.02432*, 2017.

[9] S. Cai, S. Zhou, C. Xu, and Q. Gao, "Dense motion estimation of particle images via a convolutional neural network," *Exp. Fluids*, vol. 60, no. 4, p. 73, 2019.

[10] W. R. Graham, C. W. P. Ford, and H. Babinsky, "An impulse-based approach to estimating forces in unsteady flow," *J. Fluid Mech.*, vol. 815, pp. 60–76, 2017.

[11] F. Noca, D. Shiels, and D. Jeon, "A comparison of methods for evaluating time-dependent fluid dynamic forces on bodies, using only velocity fields and their derivatives," *J. Fluids Struct.*, vol. 13, no. 5, pp. 551–578, 1999.

[12] J. Rabault, J. Kolaas, and A. Jensen, "Performing particle image velocimetry using artificial neural networks: a proof-of-concept," *Meas. Sci. Technol.*, vol. 28, no. 12, p. 125301, 2017.

[13] M. Morimoto, K. Fukami, and K. Fukagata, "Experimental velocity data estimation for imperfect particle images using machine learning," *Phys. Fluids*, vol. 33, no. 8, 2021.

[14] W. Hou, D. Darakananda, and J. Eldredge, "Machine learning based detection of flow disturbances using surface pressure measurements," in *AIAA Scitech 2019 Forum*, 2019, p. 1148.

[15] M. Le Provost, W. Hou, and J. Eldredge, "Deep learning and data assimilation approaches to sensor reduction in estimation of disturbed separated flows," in *AIAA Scitech 2020 Forum*, 2020, p. 799.

[16] Y. Zhang, W. J. Sung, and D. N. Mavris, "Application of convolutional neural network to predict airfoil lift coefficient," in *2018 AIAA/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conference*, 2018, p. 1903.

[17] J. Viquerat and E. Hachem, "A supervised neural network for drag prediction of arbitrary 2D shapes in laminar flows at low Reynolds number," *Comput. Fluids*, vol. 210, p. 104645, 2020.

[18] E. Yilmaz and B. German, "A convolutional neural network approach to training predictors for airfoil performance," in *18th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference*, 2017, p. 3660.

[19] X. Guo, W. Li, and F. Iorio, "Convolutional neural networks for steady flow approximation," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 481–490.

[20] T. P. Miyanawala and R. K. Jaiman, "An efficient deep learning technique for the Navier-Stokes equations: Application to unsteady wake flow dynamics," *arXiv preprint arXiv:1710.09099*, 2017.

[21] S. Bhatnagar, Y. Afshar, S. Pan, K. Duraisamy, and S. Kaushik, "Prediction of aerodynamic flow fields using convolutional neural networks," *Comput. Mech.*, vol. 64, no. 2, pp. 525–545, 2019.

[22] B. J. Gross, N. Trask, P. Kuberry, and P. J. Atzberger, "Meshfree methods on manifolds for hydrodynamic flows on curved surfaces: A Generalized Moving Least-Squares (GMLS) approach," *J. Comput. Phys.*, vol. 409, p. 109340, 2020.

[23] A. Hadjighasem, D. Karrasch, H. Teramoto, and G. Haller, "Spectral-clustering approach to Lagrangian vortex detection," *Phys. Rev. E*, vol. 93, no. 6, p. 063107, 2016.

[24] J. Bastings, I. Titov, W. Aziz, D. Marcheggiani, and K. Sima'an, "Graph convolutional encoders for syntax-aware neural machine translation," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 2017, pp. 1957–1967.

[25] Y. Liu, T.-P. Tan, and X. Zhan, "Iterative self-supervised learning for legal similar case retrieval," *IEEE Access*, vol. 12, pp. 17231–17241, 2024.

[26] F. Ogoke, K. Meidani, A. Hashemi, and A. B. Farimani, "Graph convolutional neural networks for body force prediction," *arXiv preprint arXiv:2012.02232*, 2020.

[27] L. Wang, Y. Fournier, J.-F. Wald, and Y. Mesri, "Identification of vortex in unstructured mesh with graph neural networks," *Comput. Fluids*, vol. 268, p. 106104, 2024.